



南開大學
Nankai University

计算机学院
深度学习实验报告

卷积神经网络实验报告

姓名：王一诺
学号：2312331
专业：计算机科学与技术

2026 年 6 月 13 日

目录

1	实验要求	2
2	实验环境与数据集	2
3	原始版本 CNN	3
3.1	网络结构	3
3.2	训练结果	4
4	ResNet	5
4.1	网络结构	5
4.2	训练结果	6
5	DenseNet	7
5.1	网络结构	7
5.2	训练结果	7
6	SE-ResNet	8
6.1	网络结构	8
6.2	训练结果	9
7	对比分析	9
7.1	最终结果对比	9
7.2	训练过程差异分析	10
8	实验心得	11

1 实验要求

- 掌握卷积的基本原理；
- 学会使用 PyTorch 搭建简单的 CNN，实现 CIFAR-10 或 CIFAR-100 数据集分类；
- 学会使用 PyTorch 搭建简单的 ResNet，实现 CIFAR-10 或 CIFAR-100 数据集分类；
- 学会使用 PyTorch 搭建简单的 DenseNet，实现 CIFAR-10 或 CIFAR-100 数据集分类；
- 学会使用 PyTorch 搭建带有 SE 模块(Squeeze-and-Excitation Networks)的 ResNet,实现 CIFAR-10 或 CIFAR-100 数据集分类；
- 对比没有跳跃连接的卷积网络、ResNet、DenseNet、SE-ResNet 在训练过程中的不同。

2 实验环境与数据集

本实验基于提供的 `cnn-pytorch-cifar10.ipynb` 进行扩展，使用 PyTorch 在 CIFAR-10 数据集上分别训练原始 CNN、ResNet、DenseNet 和 SE-ResNet。CIFAR-10 数据集包含 10 个类别，每张图像为 $3 \times 32 \times 32$ 彩色图像。实验中使用训练集进行参数更新，使用测试集作为统一验证集观察不同网络的训练 loss 和验证准确率变化。

主要运行环境如下：

```
Python: 3.10
PyTorch: 2.7.x + CUDA
Device: cuda:0
Dataset: CIFAR-10
Batch size: 4
Optimizer: SGD
Loss function: CrossEntropyLoss
Epochs: 10
```

四个模型均使用相同的 `trainloader` 和 `testloader`，从而保证验证集一致。训练循环的主要伪代码如下：

```
1 for epoch in range(num_epochs):
2     model.train()
3     epoch_loss = 0.0
4     for inputs, labels in trainloader:
5         inputs, labels = inputs.to(device), labels.to(device)
6         optimizer.zero_grad()
7         outputs = model(inputs)
8         loss = criterion(outputs, labels)
9         loss.backward()
10        optimizer.step()
11        epoch_loss += loss.item()
12
13    train_losses.append(epoch_loss / len(trainloader))
```

```

14
15 model.eval()
16 correct, total = 0, 0
17 with torch.no_grad():
18     for images, labels in testloader:
19         images, labels = images.to(device), labels.to(device)
20         outputs = model(images)
21         _, predicted = torch.max(outputs.data, 1)
22         total += labels.size(0)
23         correct += (predicted == labels).sum().item()
24 val_accuracies.append(100 * correct / total)
25

```

3 原始版本 CNN

3.1 网络结构

原始 CNN 使用两个卷积层、两个最大池化层和三个全连接层。输入图像大小为 $3 \times 32 \times 32$ ，第一层卷积将通道数从 3 变为 6，第二层卷积将通道数从 6 变为 16，最后通过全连接层输出 10 类分类结果。

核心代码如下：

```

1 class Net(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.conv1 = nn.Conv2d(3, 6, 5)
5         self.pool = nn.MaxPool2d(2, 2)
6         self.conv2 = nn.Conv2d(6, 16, 5)
7         self.fc1 = nn.Linear(16 * 5 * 5, 120)
8         self.fc2 = nn.Linear(120, 84)
9         self.fc3 = nn.Linear(84, 10)
10
11     def forward(self, x):
12         x = self.pool(F.relu(self.conv1(x)))
13         x = self.pool(F.relu(self.conv2(x)))
14         x = torch.flatten(x, 1)
15         x = F.relu(self.fc1(x))
16         x = F.relu(self.fc2(x))
17         x = self.fc3(x)
18         return x
19
20 device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')
21 net = Net().to(device)
22 print(net)
23

```

单元格输出的网络结构如下：

```
Using device: cuda:0
Net(
  (conv1): Conv2d(3, 6, kernel_size=(5, 5), stride=(1, 1))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (conv2): Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1))
  (fc1): Linear(in_features=400, out_features=120, bias=True)
  (fc2): Linear(in_features=120, out_features=84, bias=True)
  (fc3): Linear(in_features=84, out_features=10, bias=True)
)
```

3.2 训练结果

原始 notebook 中只打印了训练过程中的 loss 和最终准确率，没有曲线图。为满足实验要求，我在训练循环中增加了 `train_losses` 和 `test accuracies` 两个列表，用于记录每个 epoch 的平均训练 loss 和验证准确率，并将训练轮数设为 10 轮，使曲线具有可观察性。

关键输出如下：

```
Epoch 1: train loss = 1.710, test accuracy = 46.78%
Epoch 2: train loss = 1.344, test accuracy = 52.53%
Epoch 3: train loss = 1.199, test accuracy = 58.34%
Epoch 4: train loss = 1.104, test accuracy = 60.65%
Epoch 5: train loss = 1.036, test accuracy = 61.83%
Epoch 6: train loss = 0.979, test accuracy = 63.95%
Epoch 7: train loss = 0.929, test accuracy = 62.69%
Epoch 8: train loss = 0.894, test accuracy = 61.42%
Epoch 9: train loss = 0.860, test accuracy = 62.20%
Epoch 10: train loss = 0.830, test accuracy = 62.78%
Finished Training
```

最终在 10000 张测试图像上的准确率为：

```
Accuracy of the network on the 10000 test images: 62 %
```

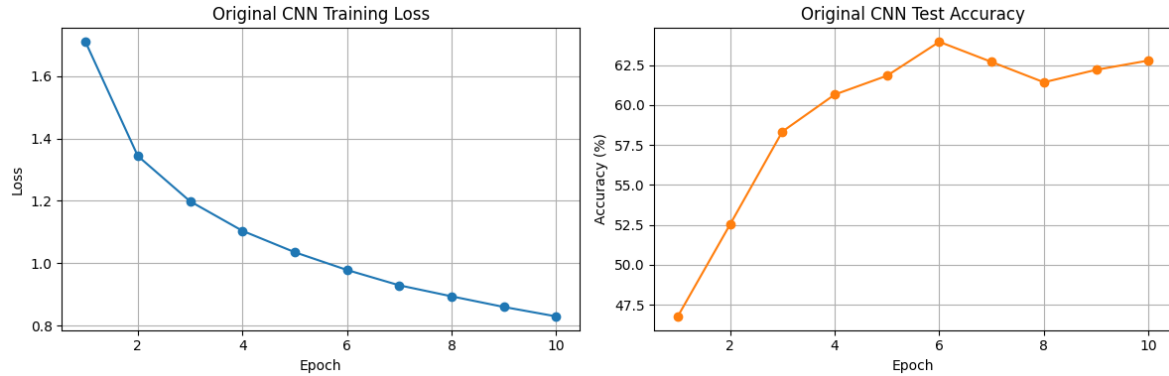


图 3.1: 原始 CNN 的训练 loss 曲线和验证准确率曲线

从图中可以看到，原始 CNN 的训练 loss 基本持续下降，但验证准确率在第 6 轮达到约 63.95% 后出现波动，最终为 62.78%。这说明简单 CNN 能够学习到 CIFAR-10 的基本图像特征，但由于网络深度较浅、表达能力有限，准确率提升空间较大。

4 ResNet

4.1 网络结构

ResNet 的核心思想是引入残差连接，将卷积层学习的映射变为 $F(x) + x$ 。这样可以缓解深层网络训练中的梯度消失和退化问题，使网络更容易优化。

本实验实现了适用于 CIFAR-10 的轻量 ResNet，包含 3 个 stage，每个 stage 有 2 个 BasicBlock。代码结构如下：

```

1 class BasicBlock(nn.Module):
2     expansion = 1
3
4     def __init__(self, in_channels, out_channels, stride=1):
5         super().__init__()
6         self.conv1 = nn.Conv2d(in_channels, out_channels, 3, stride, 1, bias=False)
7         self.bn1 = nn.BatchNorm2d(out_channels)
8         self.conv2 = nn.Conv2d(out_channels, out_channels, 3, 1, 1, bias=False)
9         self.bn2 = nn.BatchNorm2d(out_channels)
10
11        self.shortcut = nn.Sequential()
12        if stride != 1 or in_channels != out_channels:
13            self.shortcut = nn.Sequential(
14                nn.Conv2d(in_channels, out_channels, 1, stride, bias=False),
15                nn.BatchNorm2d(out_channels),
16            )
17
18        def forward(self, x):
19            out = F.relu(self.bn1(self.conv1(x)))
20            out = self.bn2(self.conv2(out))
21            out = out + self.shortcut(x)
22            out = F.relu(out)

```

```

23     return out
24

```

ResNet 主体结构如下:

```

1 class ResNetCIFAR(nn.Module):
2     def __init__(self, block=BasicBlock, num_blocks=(2, 2, 2), num_classes=10):
3         super().__init__()
4         self.in_channels = 16
5         self.conv1 = nn.Conv2d(3, 16, 3, 1, 1, bias=False)
6         self.bn1 = nn.BatchNorm2d(16)
7         self.layer1 = self._make_layer(block, 16, num_blocks[0], stride=1)
8         self.layer2 = self._make_layer(block, 32, num_blocks[1], stride=2)
9         self.layer3 = self._make_layer(block, 64, num_blocks[2], stride=2)
10        self.avg_pool = nn.AdaptiveAvgPool2d((1, 1))
11        self.fc = nn.Linear(64, num_classes)
12

```

4.2 训练结果

ResNet 使用 SGD 优化器, 学习率为 0.1, 动量为 0.9, 权重衰减为 5×10^{-4} , 并在第 5、8 轮后降低学习率。关键输出如下:

```

ResNet epoch 1: train loss = 2.052, validation accuracy = 27.86%
ResNet epoch 2: train loss = 1.882, validation accuracy = 22.11%
ResNet epoch 3: train loss = 1.773, validation accuracy = 35.46%
ResNet epoch 4: train loss = 1.736, validation accuracy = 39.82%
ResNet epoch 5: train loss = 1.721, validation accuracy = 38.10%
ResNet epoch 6: train loss = 1.215, validation accuracy = 62.83%
ResNet epoch 7: train loss = 1.092, validation accuracy = 66.58%
ResNet epoch 8: train loss = 1.016, validation accuracy = 68.78%
ResNet epoch 9: train loss = 0.777, validation accuracy = 76.08%
ResNet epoch 10: train loss = 0.712, validation accuracy = 76.37%
Finished ResNet Training

```

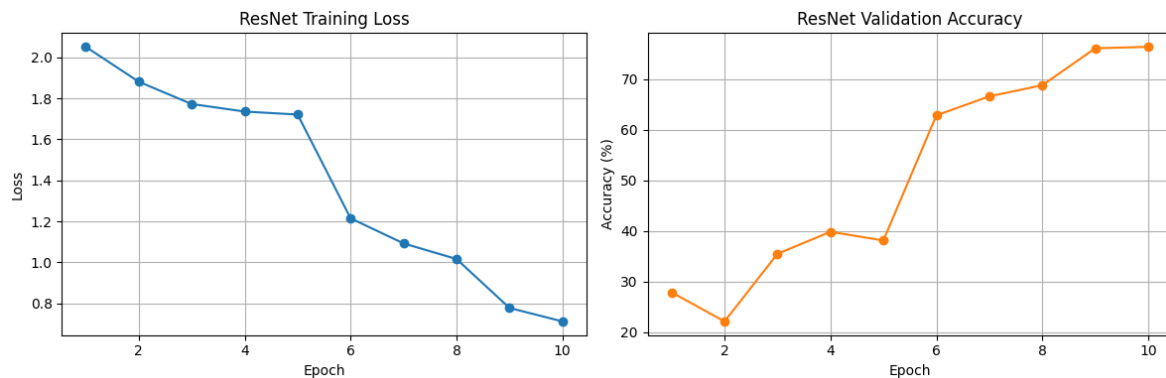


图 4.2: ResNet 的训练 loss 曲线和验证准确率曲线

ResNet 的前几轮准确率波动较大，主要是因为初始学习率较高，模型仍在寻找较优参数区域。第 6 轮学习率下降后，loss 明显下降，验证准确率迅速提升；最终准确率达到 76.37%，明显高于原始 CNN。

5 DenseNet

5.1 网络结构

DenseNet 的核心思想是密集连接：每一层的输入不仅来自上一层，还来自同一 DenseBlock 中所有之前层的特征图。其连接形式可以表示为：

$$x_l = H_l([x_0, x_1, \dots, x_{l-1}])$$

其中 $[\cdot]$ 表示通道维度上的拼接。密集连接可以增强特征复用，并改善梯度传播。

本实验实现了由 3 个 DenseBlock 和 2 个 Transition 层组成的轻量 DenseNet。核心代码如下：

```
1 class DenseLayer(nn.Module):
2     def __init__(self, in_channels, growth_rate):
3         super().__init__()
4         self.bn1 = nn.BatchNorm2d(in_channels)
5         self.conv1 = nn.Conv2d(in_channels, 4 * growth_rate, 1, bias=False)
6         self.bn2 = nn.BatchNorm2d(4 * growth_rate)
7         self.conv2 = nn.Conv2d(4 * growth_rate, growth_rate, 3, padding=1, bias=False)
8
9     def forward(self, x):
10        out = self.conv1(F.relu(self.bn1(x)))
11        out = self.conv2(F.relu(self.bn2(out)))
12        out = torch.cat([x, out], dim=1)
13        return out
14
```

5.2 训练结果

DenseNet 的训练输出如下：

```
DenseNet epoch 1: train loss = 1.956, validation accuracy = 43.77%
DenseNet epoch 2: train loss = 1.575, validation accuracy = 52.66%
DenseNet epoch 3: train loss = 1.421, validation accuracy = 55.27%
DenseNet epoch 4: train loss = 1.357, validation accuracy = 57.89%
DenseNet epoch 5: train loss = 1.310, validation accuracy = 58.36%
DenseNet epoch 6: train loss = 0.909, validation accuracy = 71.29%
DenseNet epoch 7: train loss = 0.822, validation accuracy = 73.02%
DenseNet epoch 8: train loss = 0.787, validation accuracy = 75.29%
DenseNet epoch 9: train loss = 0.672, validation accuracy = 78.71%
DenseNet epoch 10: train loss = 0.640, validation accuracy = 78.97%
Finished DenseNet Training
```

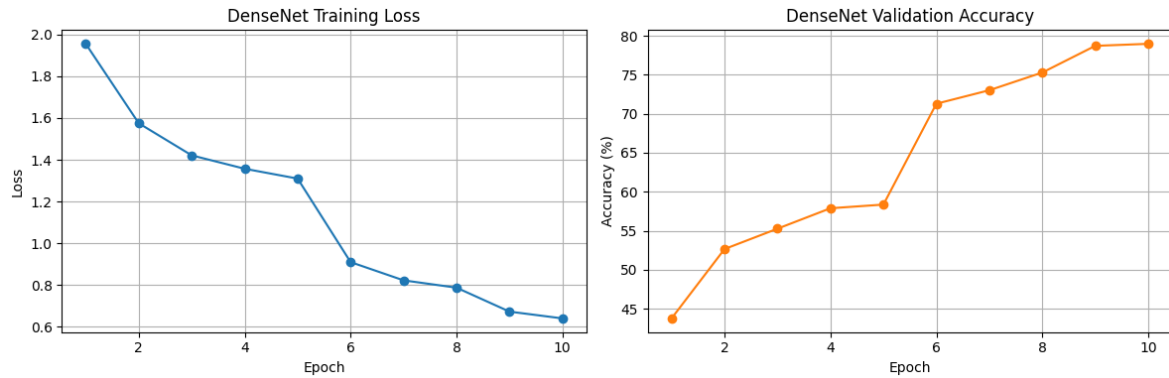


图 5.3: DenseNet 的训练 loss 曲线和验证准确率曲线

DenseNet 在第 6 轮学习率下降后准确率明显提升，并最终达到 78.97%，是本次实验中四种模型里最高的结果。相比 ResNet，DenseNet 通过特征拼接实现了更强的特征复用，因此在较少 epoch 下也能取得较好表现。

6 SE-ResNet

6.1 网络结构

SE-ResNet 在 ResNet 的残差块中加入 Squeeze-and-Excitation 模块。SE 模块首先通过全局平均池化对每个通道进行压缩（Squeeze），然后通过两层全连接网络学习通道权重（Excitation），最后用该权重重新标定各通道特征。

SE 模块代码如下：

```

1 class SEBlock(nn.Module):
2     def __init__(self, channels, reduction=4):
3         super().__init__()
4         hidden_channels = max(channels // reduction, 4)
5         self.avg_pool = nn.AdaptiveAvgPool2d(1)
6         self.fc = nn.Sequential(
7             nn.Linear(channels, hidden_channels, bias=False),
8             nn.ReLU(inplace=True),
9             nn.Linear(hidden_channels, channels, bias=False),
10            nn.Sigmoid(),
11        )
12
13    def forward(self, x):
14        batch_size, channels, _, _ = x.size()
15        weights = self.avg_pool(x).view(batch_size, channels)
16        weights = self.fc(weights).view(batch_size, channels, 1, 1)
17        return x * weights
18

```

6.2 训练结果

SE-ResNet 的训练输出如下：

```
SE-ResNet epoch 1: train loss = 1.968, validation accuracy = 34.81%
SE-ResNet epoch 2: train loss = 1.836, validation accuracy = 37.52%
SE-ResNet epoch 3: train loss = 1.776, validation accuracy = 31.07%
SE-ResNet epoch 4: train loss = 1.740, validation accuracy = 38.50%
SE-ResNet epoch 5: train loss = 1.725, validation accuracy = 41.09%
SE-ResNet epoch 6: train loss = 1.274, validation accuracy = 61.50%
SE-ResNet epoch 7: train loss = 1.165, validation accuracy = 62.06%
SE-ResNet epoch 8: train loss = 1.072, validation accuracy = 61.75%
SE-ResNet epoch 9: train loss = 0.819, validation accuracy = 75.12%
SE-ResNet epoch 10: train loss = 0.746, validation accuracy = 76.72%
Finished SE-ResNet Training
```

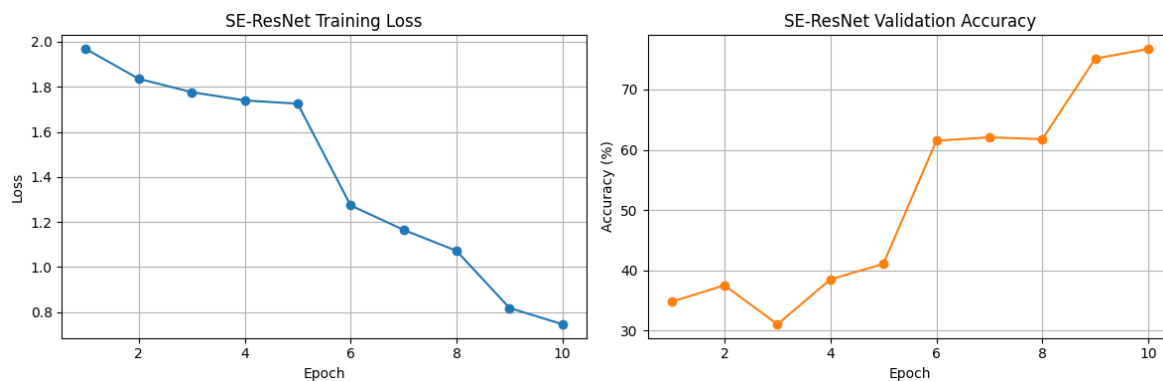


图 6.4: SE-ResNet 的训练 loss 曲线和验证准确率曲线

SE-ResNet 的最终验证准确率为 76.72%，略高于本次 ResNet 的 76.37%，但提升幅度很小。前期准确率波动较大，说明 SE 模块虽然增强了通道重标定能力，但也增加了需要学习的门控参数。在 10 个 epoch 的短训练设置下，SE 模块的优势并不一定充分体现。

7 对比分析

7.1 最终结果对比

四种模型的最终训练结果如下表所示：

表 1: 四种网络最终训练结果对比

模型	第 10 轮训练 loss	第 10 轮验证准确率
原始 CNN	0.830	62.78%
ResNet	0.712	76.37%
DenseNet	0.640	78.97%
SE-ResNet	0.746	76.72%

最终对比曲线如下：

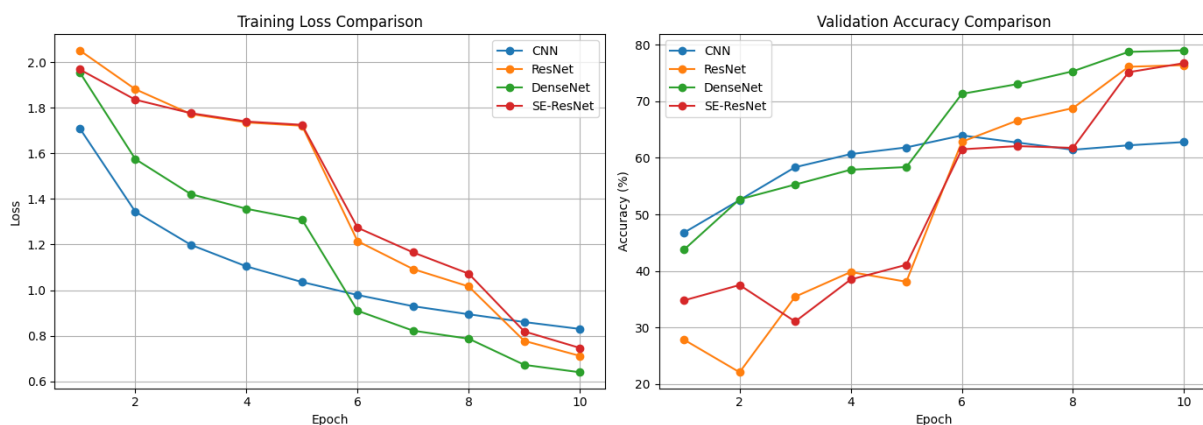


图 7.5: 原始 CNN、ResNet、DenseNet、SE-ResNet 的 loss 与准确率对比

生成对比图的代码如下：

```
1 model_curves = {
2     'CNN': (train_losses, test_accuracies),
3     'ResNet': (resnet_train_losses, resnet_val_accuracies),
4     'DenseNet': (densenet_train_losses, densenet_val_accuracies),
5     'SE-ResNet': (seresnet_train_losses, seresnet_val_accuracies),
6 }
7
8 plt.figure(figsize=(14, 5))
9 plt.subplot(1, 2, 1)
10 for model_name, (losses, _) in model_curves.items():
11     epochs = range(1, len(losses) + 1)
12     plt.plot(epochs, losses, marker='o', label=model_name)
13 plt.title('Training Loss Comparison')
14 plt.legend()
15
16 plt.subplot(1, 2, 2)
17 for model_name, (_, accuracies) in model_curves.items():
18     epochs = range(1, len(accuracies) + 1)
19     plt.plot(epochs, accuracies, marker='o', label=model_name)
20 plt.title('Validation Accuracy Comparison')
21 plt.legend()
22 plt.show()
23
```

7.2 训练过程差异分析

原始 CNN 没有跳跃连接，特征从前向后逐层传递。其优点是结构简单、训练速度快，但缺点是表达能力有限，随着网络加深容易出现梯度传播困难。本实验中原始 CNN 的 loss 能够稳定下降，但验证准确率在 60% 左右波动，最终为 62.78%，说明浅层 CNN 对 CIFAR-10 的复杂图像特征刻画不足。

ResNet 通过残差连接将输入直接加到卷积变换后的输出上，使网络学习残差函数 $F(x)$ 而不是完

整映射 $H(x)$ 。这种结构改善了梯度传播，使更深网络更容易训练。本实验中 ResNet 在学习率下降后准确率快速提升，最终达到 76.37%，相比原始 CNN 提升明显。

DenseNet 使用密集连接，每一层都能够接收之前所有层的特征。相比 ResNet 的相加连接，DenseNet 采用通道拼接，保留了更多历史特征，有利于特征复用和梯度传播。本实验中 DenseNet 最终达到 78.97%，训练 loss 最低，说明密集连接在本任务上取得了较好的效果。

SE-ResNet 在 ResNet 的基础上加入通道注意力机制，能够根据输入样本动态调整不同通道的重要性。理论上，SE 模块可以增强有效特征通道、抑制无效通道。但在本实验中，SE-ResNet 最终准确率为 76.72%，只比 ResNet 略高。原因可能包括：训练轮数只有 10 轮，SE 模块增加了额外参数和优化难度；CIFAR-10 图像尺寸较小，轻量网络通道数较少，通道注意力的优势不一定充分体现；单次实验也存在随机初始化和 batch 顺序带来的波动。

综合来看，跳跃连接和密集连接都显著优于无跳跃连接的原始 CNN；DenseNet 在本次实验中表现最好；SE-ResNet 相比 ResNet 有轻微提升，但提升不明显，需要更长训练、更充分的数据增强和多次重复实验才能更稳定地评价 SE 模块的贡献。

8 实验心得

通过本次实验，我掌握了使用 PyTorch 在 CIFAR-10 数据集上搭建和训练卷积神经网络的方法。原始 CNN 结构简单，能够完成基本分类任务，但准确率较低；ResNet 通过残差连接明显改善了深层网络的训练效果；DenseNet 通过密集连接进一步增强了特征复用能力，在本实验中取得最好结果；SE-ResNet 引入通道注意力机制，能够对特征通道进行动态重标定，但在短训练和小模型设置下提升有限。

本实验也说明，模型结构的理论优势需要结合训练策略、数据增强、学习率调度和训练轮数共同发挥作用。仅增加模块并不一定立即带来显著提升，必须通过曲线观察训练过程，并结合 loss、验证准确率和模型结构进行综合分析。